

Skill Session 5

Task 1

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
j_rakshitha@RAKSHITHA:~$ cd ~/ossp_project
j_rakshitha@RAKSHITHA:~/ossp_project$ pwd
/home/j_rakshitha/ossp_project
j_rakshitha@RAKSHITHA:~/ossp_project$ mkdir -p src build
j_rakshitha@RAKSHITHA:~/ossp_project$ ls
Makefile  build  copied.txt  include  sample.txt  src
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/squote_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ ls -l src/squote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 909 Aug 24 03:53 src/squote_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/squote_demo.c -o build/squote_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/squote_demo
Single Quote Parsing Demo
Single quotes preserve literal content.
Enter a quoted string: hello

Input received: hello

Parsing Result:
No valid single-quoted string detected.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/squote_demo
Single Quote Parsing Demo
Single quotes preserve literal content.
Enter a quoted string: hello world

Input received: hello world

Parsing Result:
No valid single-quoted string detected.
```

```
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/squote_demo
Single Quote Parsing Demo
Single quotes preserve literal content.
Enter a quoted string: 'hello world'

Input received: 'hello world'

Parsing Result:
Single quotes detected.
Variable expansion is ignored.
Content is preserved literally.
Literal content: hello world
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/squote_demo
Single Quote Parsing Demo
Single quotes preserve literal content.
Enter a quoted string: '$HOME'

Input received: '$HOME'

Parsing Result:
Single quotes detected.
Variable expansion is ignored.
Content is preserved literally.
Literal content: $HOME
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```
j_rakshitha@RAKSHITHA: ~  X  +  v
GNU nano 8.7.1 src/squote_demo.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];

    printf("Single Quote Parsing Demo\n");
    printf("Single quotes preserve literal content.\n");
    printf("Enter a quoted string: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    printf("\nInput received: %s\n", input);
    printf("\nParsing Result:\n");

    if (strlen(input) >= 2 &&
        input[0] == '\'' &&
        input[strlen(input) - 1] == '\'')
    {
        printf("Single quotes detected.\n");
        printf("Variable expansion is ignored.\n");
        printf("Content is preserved literally.\n");

        printf("Literal content: ");

        for (int i = 1; i < (int)strlen(input) - 1; i++)
        {
            putchar(input[i]);
        }

        printf("\n");
    }
    else
    {
        printf("No valid single-quoted string detected.\n");
    }

    return 0;
}
```

Task 2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/dquote_demo.c

j_rakshitha@RAKSHITHA:~/ossp_project$ ls -l src/dquote_demo.c
-rw-r--r-- 1 j_rakshitha j_rakshitha 1257 Aug 24 03:56 src/dquote_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/dquote_demo.c -o build/dquote_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/dquote_demo
Double Quote Parsing Demo
Double quotes preserve spaces and allow variable expansion.
Enter a double-quoted string: "hello world"

Input received: "hello world"

Parsing Result:
Double quotes detected.
Spaces are preserved.
Quoted content: hello world
No variable expansion required.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/dquote_demo
Double Quote Parsing Demo
Double quotes preserve spaces and allow variable expansion.
Enter a double-quoted string: "$HOME"

Input received: "$HOME"

Parsing Result:
Double quotes detected.
Spaces are preserved.
Quoted content: $HOME
Variable $HOME detected.
Expanded value: /home/j_rakshitha
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```

GNU nano 8.7.1 src/dquote_demo.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    char input[200];
    char *home;

    printf("Double Quote Parsing Demo\n");
    printf("Double quotes preserve spaces and allow variable expansion.\n");
    printf("Enter a double-quoted string: ");

    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nInput received: %s\n", input);
    printf("\nParsing Result:\n");

    if (strlen(input) >= 2 &&
        input[0] == '"' &&
        input[strlen(input) - 1] == '"')
    {
        printf("Double quotes detected.\n");
        printf("Spaces are preserved.\n");

        printf("Quoted content: ");

        for (int i = 1; i < (int)strlen(input) - 1; i++)
        {
            putchar(input[i]);
        }

        printf("\n");

        home = getenv("HOME");

        if (strstr(input, "$HOME") != NULL)
        {
            printf("Variable $HOME detected.\n");

            if (home != NULL)
            {
                printf("Expanded value: %s\n", home);
            }
        }
        else
        {
            printf("No variable expansion required.\n");
        }
    }
    else
    {
        printf("No valid double-quoted string detected.\n");
    }

    return 0;
}

```

Skill Session 6

Task 1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```

j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/escape_demo.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/escape_demo.c -o build/escape_demo
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/escape_demo
Escape Sequence Parsing Demo
Enter a command with escape sequences: hello\ world

Original Input : hello\ world
Parsed Output  : hello world

Escape sequence processing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/escape_demo
Escape Sequence Parsing Demo
Enter a command with escape sequences: hello\ \$HOME

Original Input : hello\ \$HOME
Parsed Output  : hello $HOME

Escape sequence processing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/escape_demo
Escape Sequence Parsing Demo
Enter a command with escape sequences: say\ \"hello\"

Original Input : say\ \"hello\"
Parsed Output  : say "hello"

Escape sequence processing completed successfully.
j_rakshitha@RAKSHITHA:~/ossp_project$ |

```

```

GNU nano 8.7.1 src/escape_demo.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[300];
    char output[300];
    int i = 0;
    int j = 0;

    printf("Escape Sequence Parsing Demo\n");
    printf("Enter a command with escape sequences: ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0')
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;
            output[j++] = input[i];
        }
        else
        {
            output[j++] = input[i];
        }

        i++;
    }

    output[j] = '\0';

    printf("\nOriginal Input : %s\n", input);
    printf("Parsed Output  : %s\n", output);

    printf("\nEscape sequence processing completed successfully.\n");

    return 0;
}

```

Task 2

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
j_rakshitha@RAKSHITHA:~/ossp_project$ nano src/child_exec.c
j_rakshitha@RAKSHITHA:~/ossp_project$ gcc -Wall -Wextra -g src/child_exec.c -o build/child_exec
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_exec ls
Parent Process Started
Parent PID: 878

Parent waiting for child...

Child Process Created
Child PID: 879
Executing command: ls

Makefile build copied.txt include sample.txt src

Child completed with exit status: 0
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_exec ls -l
Parent Process Started
Parent PID: 889

Parent waiting for child...

Child Process Created
Child PID: 890
Executing command: ls

total 24
-rw-r--r-- 1 j_rakshitha j_rakshitha 164 Aug 18 04:37 Makefile
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 24 04:02 build
-rw-r--r-- 1 j_rakshitha j_rakshitha 51 Aug 18 05:21 copied.txt
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 17 05:28 include
-rw-r--r-- 1 j_rakshitha j_rakshitha 51 Aug 18 05:20 sample.txt
drwxr-xr-x 2 j_rakshitha j_rakshitha 4096 Aug 24 04:02 src

Child completed with exit status: 0
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ ./build/child_exec nonexistentcommand
Parent Process Started
Parent PID: 902

Parent waiting for child...

Child Process Created
Child PID: 903
Executing command: nonexistentcommand

execvp failed: No such file or directory

Child completed with exit status: 1
Parent process exiting.
j_rakshitha@RAKSHITHA:~/ossp_project$ |
```

```

GNU nano 8.7.1 src/child_exec.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process Created\n");
        printf("Child PID: %d\n", getpid());
        printf("Executing command: %s\n\n", argv[1]);

        execvp(argv[1], &argv[1]);

        perror("execvp failed");
        exit(1);
    }

    printf("\nParent waiting for child...\n");

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("\nChild completed with exit status: %d\n",
            WEXITSTATUS(status));
    }

    printf("Parent process exiting.\n");

    return 0;
}

```

Practice Tasks

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
j_rakshitha@RAKSHITHA:~$ pwd
/home/j_rakshitha
j_rakshitha@RAKSHITHA:~$ nano task1.c
j_rakshitha@RAKSHITHA:~$ gcc task1.c -o task1
j_rakshitha@RAKSHITHA:~$ ./task1
myshell> hello
You entered: hello
myshell> testing
You entered: testing
myshell> exit
Exiting shell...
j_rakshitha@RAKSHITHA:~$ echo "This is the content of File1." > File1.txt
j_rakshitha@RAKSHITHA:~$ cat File1.txt
This is the content of File1.
j_rakshitha@RAKSHITHA:~$ nano task2.c
j_rakshitha@RAKSHITHA:~$ gcc task2.c -o task2
j_rakshitha@RAKSHITHA:~$ ./task2
File copied successfully.
j_rakshitha@RAKSHITHA:~$ cat File2.txt
This is the content of File1.
j_rakshitha@RAKSHITHA:~$ ls -l File1.txt File2.txt
-rw-r--r-- 1 j_rakshitha j_rakshitha 30 Aug 12 06:40 File1.txt
-rw-r--r-- 1 j_rakshitha j_rakshitha 30 Aug 12 06:41 File2.txt
j_rakshitha@RAKSHITHA:~$ nano task3.c
j_rakshitha@RAKSHITHA:~$ gcc task3.c -o task3
j_rakshitha@RAKSHITHA:~$ ./task3
mysh> hello
Command: hello
mysh> testing
Command: testing
mysh> exit
j_rakshitha@RAKSHITHA:~$ ./task3
mysh> hell
Command: hell
mysh> exit
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

    while (1) {
        printf("myshell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}

```

```

GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    ssize_t bytesRead, bytesWritten;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 == -1) {
        perror("Error opening File1.txt");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 == -1) {
        perror("Error opening File2.txt");
        close(fd1);
        return 1;
    }

    while ((bytesRead = read(fd1, buffer, sizeof(buffer))) > 0) {
        bytesWritten = write(fd2, buffer, bytesRead);

        if (bytesWritten == -1) {
            perror("Error writing File2.txt");
            close(fd1);
            close(fd2);
            return 1;
        }
    }
}

```

```
    if (bytesRead == -1) {  
        perror("Error reading File1.txt");  
    }  
  
    close(fd1);  
    close(fd2);  
  
    printf("File copied successfully.\n");  
  
    return 0;  
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K
^U

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <termios.h>

#define MAX 100

int main() {
    char buffer[MAX];
    char ch;
    int index;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {

        write(STDOUT_FILENO, "mysh> ", 6);

        index = 0;

        while (1) {

            read(STDIN_FILENO, &ch, 1);

            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';
                write(STDOUT_FILENO, "\n", 1);
                break;
            }
        }
    }
}
```

```

        break;
    }

    if ((ch == 127 || ch == 8) && index > 0) {
        index--;
        write(STDOUT_FILENO, "\b \b", 3);
    }

    else if (index < MAX - 1) {
        buffer[index++] = ch;
        write(STDOUT_FILENO, &ch, 1);
    }
}

if (strcmp(buffer, "exit") == 0) {
    break;
}

write(STDOUT_FILENO, "Command: ", 9);
write(STDOUT_FILENO, buffer, strlen(buffer));
write(STDOUT_FILENO, "\n", 1);
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}
|

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;

void add_history(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void clear_line(int length) {
    for (int i = 0; i < length; i++) {
        write(STDOUT_FILENO, "\b \b", 3);
    }
}

int main() {

```

Using the following system calls, copy the content from File1.txt to File2.txt.

1. Open()
2. Read()
3. Write()
4. Close()
5. open()

```
j_rakshitha@RAKSHITHA:~$ nano task4.c
j_rakshitha@RAKSHITHA:~$ gcc task4.c -o task4
j_rakshitha@RAKSHITHA:~$ ./task4
myshell> pwd
/home/j_rakshitha
myshell> ls
File1.txt  command_executor  task1.c  task3  task4.c
File2.txt  command_executor.c task2    task3.c
OSSP      task1             task2.c  task4
myshell> ls
File1.txt  command_executor  task1.c  task3  task4.c
File2.txt  command_executor.c task2    task3.c
OSSP      task1             task2.c  task4
myshell> exit
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_HISTORY 10

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;

void add_history(char *command) {
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY) {
        strcpy(history[history_count], command);
        history_count++;
    } else {
        for (int i = 0; i < MAX_HISTORY - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

void clear_line(int length) {
    for (int i = 0; i < length; i++) {
        write(STDOUT_FILENO, "\b \b", 3);
    }
}

int main() {
```

```
int main() {

    struct termios oldt, newt;
    char buffer[MAX_INPUT];
    char ch;

    tcgetattr(STDIN_FILENO, &oldt);

    newt = oldt;
    newt.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        int index = 0;
        int history_position = history_count;

        while (1) {

            read(STDIN_FILENO, &ch, 1);

            if (ch == '\n' || ch == '\r') {
                buffer[index] = '\0';
                printf("\n");
                break;
            }

            if (ch == 127 || ch == 8) {
                if (index > 0) {
                    index--;
                    printf("\b \b");
                    fflush(stdout);
                }
            }
        }
    }
}
```

```
        fflush(stdout);
    }
    continue;
}

if (ch == 27) {

    char seq[2];

    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);

    if (seq[0] == '[' && seq[1] == 'A') {

        if (history_position > 0) {
            history_position--;

            clear_line(index);

            strcpy(buffer, history[history_position]);
            index = strlen(buffer);

            printf("%s", buffer);
            fflush(stdout);
        }

    } else if (seq[0] == '[' && seq[1] == 'B') {

        if (history_position < history_count - 1) {

            history_position++;

            clear_line(index);

            strcpy(buffer, history[history_position]);
            index = strlen(buffer);
```

```
    } else if (seq[0] == '[' && seq[1] == 'B') {  
  
        if (history_position < history_count - 1) {  
  
            history_position++;  
  
            clear_line(index);  
  
            strcpy(buffer, history[history_position]);  
            index = strlen(buffer);  
  
            printf("%s", buffer);  
            fflush(stdout);  
  
        } else if (history_position == history_count - 1) {  
  
            history_position = history_count;  
  
            clear_line(index);  
  
            index = 0;  
            buffer[0] = '\\0';  
  
            fflush(stdout);  
        }  
    }  
  
    continue;  
}  
  
if (index < MAX_INPUT - 1) {  
    buffer[index++] = ch;  
    write(STDOUT_FILENO, &ch, 1);  
}  
  
if (strcmp(buffer, "exit") == 0)
```

```
    }

    if (strcmp(buffer, "exit") == 0)
        break;

    add_history(buffer);

    if (strlen(buffer) == 0)
        continue;

    char *args[MAX_INPUT];
    int argc = 0;

    char *token = strtok(buffer, " ");

    while (token != NULL && argc < MAX_INPUT - 1) {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    pid_t pid = fork();

    if (pid == 0) {
        execvp(args[0], args);

        perror("Command not found");
        exit(1);
    } else if (pid > 0) {
        wait(NULL);
    } else {
```

```
char *args[MAX_INPUT];
int argc = 0;

char *token = strtok(buffer, " ");

while (token != NULL && argc < MAX_INPUT - 1) {
    args[argc++] = token;
    token = strtok(NULL, " ");
}

args[argc] = NULL;

pid_t pid = fork();

if (pid == 0) {
    execvp(args[0], args);

    perror("Command not found");
    exit(1);
} else if (pid > 0) {
    wait(NULL);
} else {
    perror("fork failed");
}

tcsetattr(STDIN_FILENO, TCSANOW, &oldt);

return 0;
}
```

```

j_rakshitha@RAKSHITHA:~$ nano task5.c
j_rakshitha@RAKSHITHA:~$ gcc task5.c -o task5
task5.c: In function 'main':
task5.c:42:28: error: implicit declaration of function 'mremap'; did you mean
n 'munmap'? [-Wimplicit-function-declaration]
    42 |         char *new_buffer = mremap(
        |                             ^~~~~~
        |                             munmap
task5.c:46:13: error: 'MREMAP_MAYMOVE' undeclared (first use in this function)
    46 |         MREMAP_MAYMOVE
        |         ^~~~~~
task5.c:46:13: note: each undeclared identifier is reported only once for each
function it appears in
j_rakshitha@RAKSHITHA:~$ gcc -D_GNU_SOURCE task5.c -o task5
j_rakshitha@RAKSHITHA:~$ ./task5
Enter a command: hello world

Command stored in linked list: hello world
Memory allocated successfully.
Releasing memory...
Memory released successfully.
j_rakshitha@RAKSHITHA:~$ gcc -D_GNU_SOURCE task5.c -o task5
j_rakshitha@RAKSHITHA:~$

```

```

Memory released successfully.
j_rakshitha@RAKSHITHA:~$ gcc -D_GNU_SOURCE task5.c -o task5
j_rakshitha@RAKSHITHA:~$ valgrind --leak-check=full ./task5
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
j_rakshitha@RAKSHITHA:~$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages
s [376 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages
[454 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-e
n [91.4 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Compone
nts [32.7 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Pac
kages [154 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe Translat
ion-en [49.8 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Co
mponents [43.2 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en
[112 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Componen
ts [84.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Pack
ages [237 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translatio
n-en [77.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Comp
onents [183 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Pa

```

```
j_rakshitha@RAKSHITHA:~$ valgrind --version
valgrind-3.26.0
j_rakshitha@RAKSHITHA:~$ valgrind --leak-check=full ./task5
==1285== Memcheck, a memory error detector
==1285== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1285== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1285== Command: ./task5
==1285==
Enter a command: hii

Command stored in linked list: hii
Memory allocated successfully.
Releasing memory...
Memory released successfully.
==1285==
==1285== HEAP SUMMARY:
==1285==     in use at exit: 0 bytes in 0 blocks
==1285==   total heap usage: 4 allocs, 4 frees, 2,068 bytes allocated
==1285==
==1285== All heap blocks were freed -- no leaks are possible
==1285==
==1285== For lists of detected and suppressed errors, rerun with: -s
==1285== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
j_rakshitha@RAKSHITHA:~$
```

```
ckages [334 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translat
ion-en [63.4 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Pa
ckages [11.3 kB]
Fetched 2578 kB in 16s (157 kB/s)
40 packages can be upgraded. Run 'apt list --upgradable' to see them.
j_rakshitha@RAKSHITHA:~$ sudo apt install valgrind
Installing:
  valgrind

Installing dependencies:
  gdb                libdebuginfod-common  libipt2
  libbabeltrace1     libdebuginfod1t64      libsource-highlight-common
  libc6-dbg          libdw1t64              libsource-highlight4t64

Suggested packages:
  gdb-doc  gdbserver  valgrind-mpi  kcachegrind

Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 40
  Download size: 27.1 MB
  Space needed: 111 MB / 1024 GB available

Continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdebuginfod-com
mon all 0.194-4 [12.3 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdw1t64 amd64 0
.194-4 [285 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute/main amd64 libbabeltrace1 am
d64 1.5.11-5build1 [165 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute/main amd64 libdebuginfod1t64
amd64 0.194-4 [20.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute/main amd64 libipt2 amd64 2.1
.2-3 [49.2 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute/main amd64 libsource-highlig
ht-common all 3.1.9-4.3build2 [64.2 kB]
Get:7 http://archive.ubuntu.com/ubuntu resolute/main amd64 libsource-highlig
ht4t64 amd64 3.1.9-4.3build2 [273 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/main amd64 gdb amd64 17.1-2u
buntu1 [4173 kB]
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>

#define INITIAL_SIZE 64

struct Node {
    char *command;
    struct Node *next;
};

int main() {
    size_t size = INITIAL_SIZE;

    char *buffer = mmap(
        NULL,
        size,
        PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS,
        -1,
        0
    );

    if (buffer == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    printf("Enter a command: ");

    if (fgets(buffer, size, stdin) == NULL) {
        munmap(buffer, size);
        return 1;
    }
}
```

```
if (strlen(buffer) == size - 1 &&
    buffer[size - 2] != '\n') {

    size_t new_size = size * 2;

    char *new_buffer = mremap(
        buffer,
        size,
        new_size,
        MREMAP_MAYMOVE
    );

    if (new_buffer == MAP_FAILED) {
        perror("mremap failed");
        munmap(buffer, size);
        return 1;
    }

    buffer = new_buffer;
    size = new_size;
}

buffer[strcspn(buffer, "\n")] = '\0';

struct Node *head = malloc(sizeof(struct Node));

if (head == NULL) {
    perror("malloc failed");
    munmap(buffer, size);
    return 1;
}

head->command = malloc(strlen(buffer) + 1);
```

```
    struct Node *head = malloc(sizeof(struct Node));

    if (head == NULL) {
        perror("malloc failed");
        munmap(buffer, size);
        return 1;
    }

    head->command = malloc(strlen(buffer) + 1);

    if (head->command == NULL) {
        perror("malloc failed");
        free(head);
        munmap(buffer, size);
        return 1;
    }

    strcpy(head->command, buffer);

    head->next = NULL;

    printf("\nCommand stored in linked list: %s\n", head->command);

    printf("Memory allocated successfully.\n");
    printf("Releasing memory...\n");

    free(head->command);
    free(head);

    munmap(buffer, size);

    printf("Memory released successfully.\n");

    return 0;
}
```